

Lab6 Report: Generative Models

張苙烜/ 資科工碩 313551079

1 Introduction

本次 lab 需要實作一個 Conditional Denoising Diffusion Probabilistic Model，根據 multi-label 生成對應的 image，並且根據 pretrained 的 ResNet-18 架構 classifier 評估是否包含對應的 label，測量最終的準確率。實驗中使用 Hugging Face Diffusers 套件設計 unet 架構，並且在 sample 中加入 classifier guidance 策略提升條件限制產生的結果。

2 Implementation Details

2.1 Dataset

ICLVRDataset 類別將 input normalize 成 $[-1,1]$ 之間，並且 resize 成 64x64。透過 `_labels_to_onehot` 轉換成 object.json 對應的 label。

```
1 class ICLVRDataset(Dataset):
2     def __init__(self, img_root, json_path, object_json_path, mode='train'):
3         self.img_root = img_root
4         self.mode = mode
5
6         with open(json_path, 'r') as f:
7             self.data = json.load(f)
8
9         with open(object_json_path, 'r') as f:
10             self.object_dict = json.load(f) # e.g., {"gray cube": 0, ...}
11
12         self.transform = transforms.Compose([
13             transforms.Resize((64, 64)),
14             transforms.ToTensor(),
15             transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
16         ])
17
18         if mode == 'train':
19             self.filenamees = list(self.data.keys())
20         else:
21             self.conditions = self.data
22
23     def __len__(self):
24         if self.mode == 'train':
25             return len(self.filenamees)
26         else:
27             return len(self.conditions)
28
29     def __getitem__(self, idx):
30         if self.mode == 'train':
31             img_name = self.filenamees[idx]
32             img_path = os.path.join(self.img_root, img_name)
33             img = Image.open(img_path).convert('RGB')
34
```

```

35         label_list = self.data[img_name] # ["red cube", "gray cylinder", ...]
36
37     else:
38         # test.json or new_test.json
39         img = torch.randn(3, 64, 64)
40         label_list = self.conditions[idx] # ["red sphere", "cyan cylinder", ...]
41
42     label_tensor = self._labels_to_onehot(label_list)
43
44     if self.mode == 'train':
45         img = self.transform(img)
46         label_tensor.nonzero().squeeze().tolist()
47
48     return img, label_tensor
49
50     # 根據 object.json 的 key，將 labels 轉成 one-hot vector
51     def _labels_to_onehot(self, labels):
52         onehot = torch.zeros(len(self.object_dict))
53         for obj_name in labels:
54             idx = self.object_dict[obj_name]
55             onehot[idx] = 1
56     return onehot

```

2.2 DDPM

基於 Hugging Face 的 UNet2DModel 和 DDPM Scheduler 建立架構，Unet 架構基於 UNet2DModel，noise scheduler 使用 DDPM Scheduler 控制擴散過程中的 α β 值， $\beta_t \in (0, 1)$ ， $\alpha_t = 1 - \beta_t$ ， $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ ，forward process 前向加噪過程 $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_{t-1} + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon$ ， $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ ，Reverse Process $\hat{\mathbf{x}}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \cdot \hat{\epsilon}_\theta(\mathbf{x}_t, t))$ 。

```

1 class ConditionalDDPM(nn.Module):
2     def __init__(self, model, cond_dim, timesteps=1000, device='cuda'):
3         super().__init__()
4         self.model = model
5         self.scheduler = DDPM Scheduler(timesteps)
6         self.evaluator = evaluation_model()
7         self.device = device
8         self.timesteps = timesteps
9         print(self.scheduler.config)
10
11         self.scheduler.alphas_cumprod = self.scheduler.alphas_cumprod.to(device)
12
13         self.cond_proj = nn.Linear(cond_dim, 256)
14
15     def sample_noise(self, shape):
16         return torch.randn(shape, device=self.device)
17
18     def forward_diffusion(self, x0, t):
19
20         noise = self.sample_noise(x0.shape)
21         x_t = self.scheduler.add_noise(x0, noise, t)
22         return x_t, noise

```

Conditional embedding 是透過一層線性投影 (Linear Layer) 將物件條件的 one-hot 向量 (依據 objects.json 定義的 24 類別) 嵌入成連續的向量表示，並作為 UNet 中 cross-attention 機制的 encoder_hidden_states，以引導影像生成過程符合指定條件。這邊為了強化生成影像與條件，加入了 Classifier Guidance，每五步做一次 classifier guidance，根據 pretrained 的分類器調整權重。

```

1     def predict_noise(self, x_t, t, condition):

```

```

2
3     # conditional embedding
4     emb = self.cond_proj(condition) # (B, in_channels)
5     out = self.model(x_t, t, emb).sample
6     return out
7
8 def reverse_diffusion(self, x_t, t, condition, guidance_scale=1.0):
9
10    pred_noise = self.predict_noise(x_t, t, condition)
11
12    if t[0].item() % 5 == 0:
13        with torch.enable_grad():
14            x_t.requires_grad_(True)
15            logits = self.evaluator.resnet18(x_t)
16            target_class = condition.argmax(dim=-1)
17            loss = -logits[range(len(target_class)), target_class].sum()
18            grad = torch.autograd.grad(loss, x_t)[0]
19            x_t = x_t - guidance_scale * grad
20            x_t = x_t.detach()
21
22    x_prev = self.scheduler.step(pred_noise, t, x_t).prev_sample
23    return x_prev
24
25 @torch.no_grad()
26 def p_sample_loop(self, shape, condition):
27
28    x = torch.randn(shape, device=self.device)
29    for i in reversed(range(self.scheduler.config.num_train_timesteps)):
30        t = torch.full((shape[0],), i, device=self.device, dtype=torch.long)
31        x = self.reverse_diffusion(x, t, condition)
32    return x

```

2.3 Train

在訓練階段中，模型會隨機挑選一個時間步 t ，並對原始圖片 x_0 加入對應程度的高斯雜訊，產生中間狀態 x_t ，接著，模型根據條件輸入預測 noise，並以均方誤差（MSE）衡量模型預測的噪聲與實際加入的真實噪聲之間的差距，作為 loss function。

```

1 def train(args):
2     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
3
4     # dataset & dataloader
5     dataset = ICLEVRDataset(
6         img_root=args.img_root,
7         json_path=args.json_path,
8         object_json_path=args.object_json_path,
9         mode='train'
10    )
11    dataloader = DataLoader(dataset, batch_size=args.batch_size, shuffle=True,
12                            num_workers=4)
13    # model
14    unet = UNet2DModel(
15        sample_size=64,
16        in_channels=3,
17        out_channels=3,
18        layers_per_block=2,
19        block_out_channels=(64, 128, 256, 512),
20        down_block_types=("DownBlock2D", "DownBlock2D", "DownBlock2D", "
21                          AttnDownBlock2D"),

```

```

20         up_block_types=("AttnUpBlock2D", "UpBlock2D", "UpBlock2D", "UpBlock2D"),
21         class_embed_type="identity",
22     )
23     model = ConditionalDDPM(unet, cond_dim=24, timesteps=1000, device=device).to(
24         device)
25     optimizer = optim.AdamW(model.parameters(), lr=args.lr)
26     criterion = nn.MSELoss()
27
28     # initialize wandb
29     wandb.init(project="lab6-ddpm", config=vars(args))
30     best_loss = float('inf')
31     best_model_path = os.path.join(args.save_dir, "best_model.pt")
32
33     for epoch in range(args.epochs):
34
35         model.train()
36         pbar = tqdm(dataloader)
37         total_loss = 0.0
38
39         for step, (x0, cond) in enumerate(pbar):
40             x0 = x0.to(device)
41             cond = cond.to(device)
42
43             t = torch.randint(0, model.timesteps, (x0.size(0),), device=device).long()
44             x_t, noise = model.forward_diffusion(x0, t)
45
46             pred_noise = model.predict_noise(x_t, t, cond)
47
48             loss = criterion(pred_noise, noise)
49             total_loss += loss.item()
50
51             optimizer.zero_grad()
52             loss.backward()
53             optimizer.step()
54
55             pbar.set_description(f"Epoch {epoch+1} Step {step+1} Loss: {loss.item():.4f}")
56             wandb.log({"train/loss": loss.item()}, step=epoch)
57
58         avg_loss = total_loss / len(dataloader)
59         wandb.log({"epoch/loss_avg": total_loss / len(dataloader), "epoch": epoch +
60             1}, step=epoch)
61         os.makedirs(args.save_dir, exist_ok=True)
62         if avg_loss < best_loss:
63             best_loss = avg_loss
64             torch.save(model.state_dict(), best_model_path)
65             print(f"Best model saved at epoch {epoch+1} with loss {avg_loss:.4f}")
66             torch.save(model.state_dict(), os.path.join(args.save_dir, f"ddpm_epoch{epoch
67                 +1}.pt"))

```

3 Results and Discussion

3.1 Results

根據Table 1和Table 2可以顯示 model 的準確率與圖像可視化，Figure 1顯示了 denoising 的過程，Figure 2顯示 train loss。

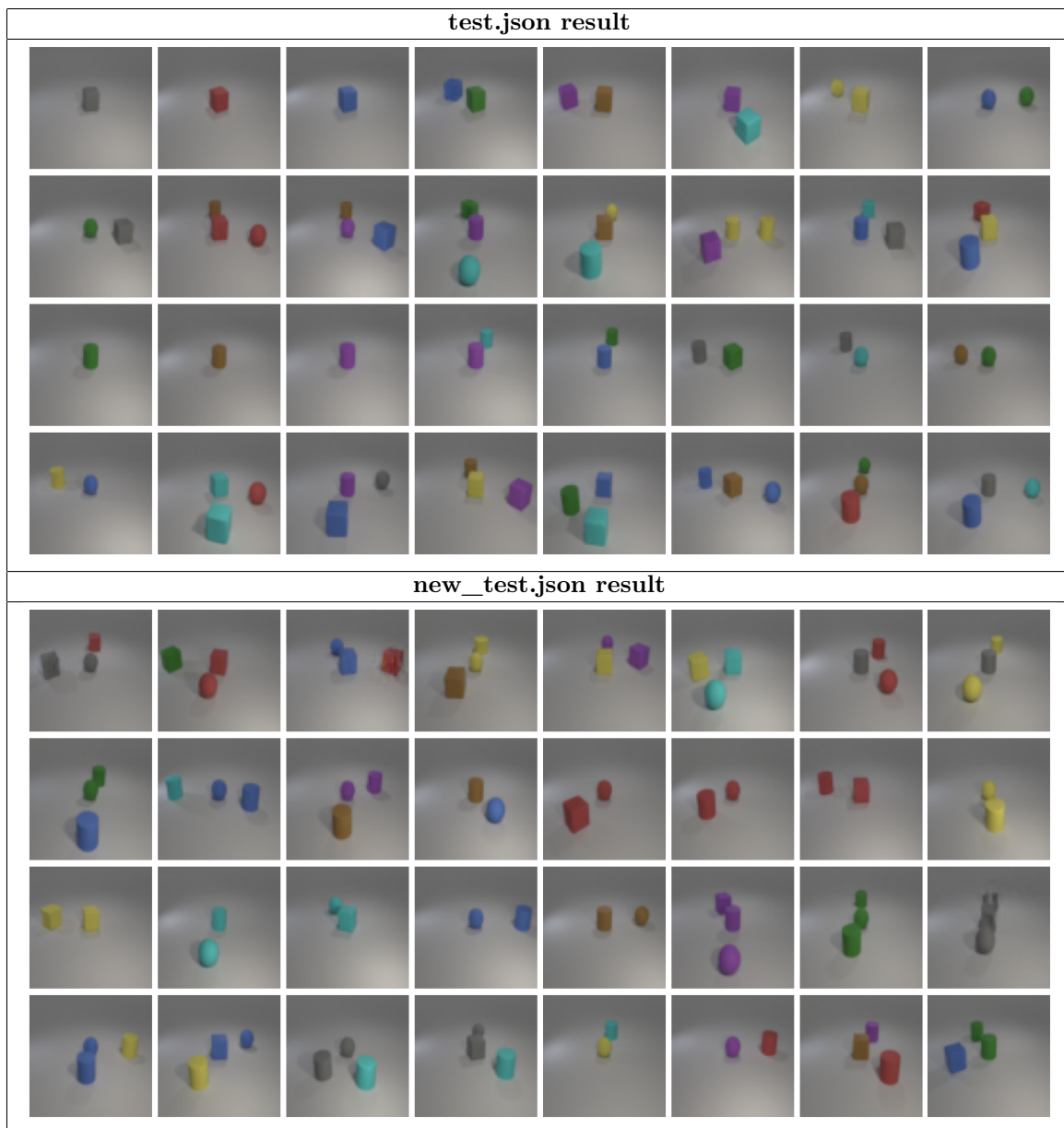


Table 1: Synthetic Image Grids

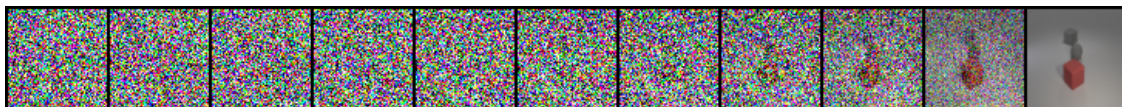


Figure 1: Example of the denoising process image

test.json result		
Generating images: 100%		32/32
Evaluation Score: 0.9306		
new_test.json result		
Generating images: 100%		32/32
Evaluation Score: 0.9524		

Table 2: Screenshot of Accuracy

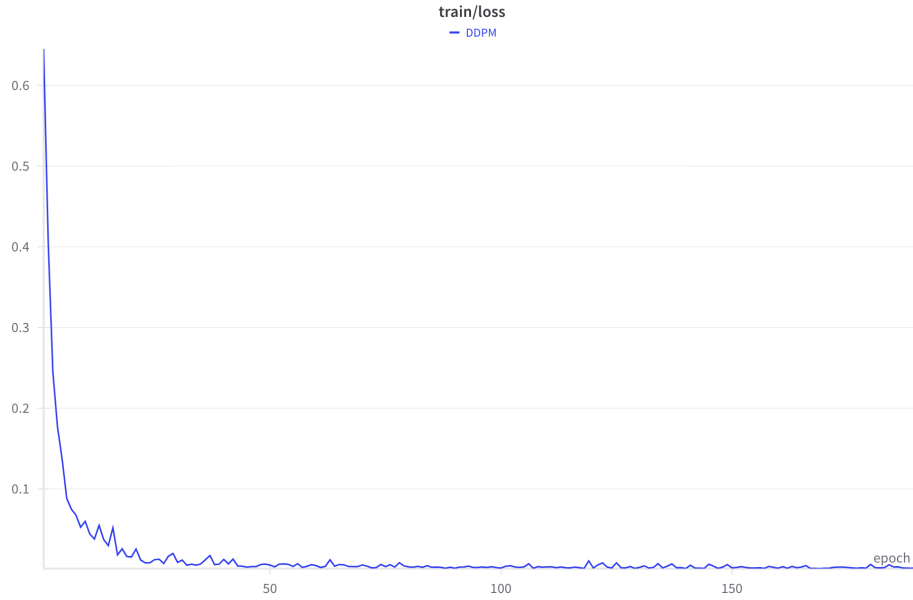


Figure 2: Train loss

3.2 Discussion

一開始我選用 Hugging face 的 `encoder_hidden_states` 作為條件輸入，藉由 cross-attention 機制將條件資訊注入 UNet 模型，但模型生成結果極度不穩定，影像內容經常與指定條件不符，測失敗的原因在於條件資訊過於簡單而與高階的 cross-attention 機制不相容，輸出維度太高太複雜。這樣 one-hot label 的資訊過於稀疏，這樣的 projection 方法反而會夾帶太多不必要的資訊，這類簡單的條件限制只需要簡單將條件丟入模型即可。

3.3 Extra experiments

為了檢驗 classifier guidance 的 generation quality, 分別對 `guidance_scale = 0,1,3,5` 分別做了 inference。從 Table 3 可以看出 classifier guidance 確實有幫助準確率的提升，但適度引導效果最佳，幫助模型生成語意更準確的圖像而不過度干擾條件限制。

guidance_scale	0	1	3	5
test	0.8472	0.9167	0.9306	0.9028
new_test	0.9286	0.9524	0.9048	0.8690

Table 3: experiment with different scale